Final Report: DT-Project LightGuard

This report presents the development and implementation of a traffic light detection system for an autonomously driving Duckiebot. The system is designed to recognize traffic lights, interpret their meanings, and respond appropriately to ensure safe and efficient navigation. The report details the modular structure of the system, including traffic light recognition, semantic understanding, and decision-making processes. Finally, we demonstrate the system's performance across diverse scenarios, highlighting its robustness and reliability.

System

To tackle the challenge of recognizing traffic lights and responding appropriately, we employ the following setup:

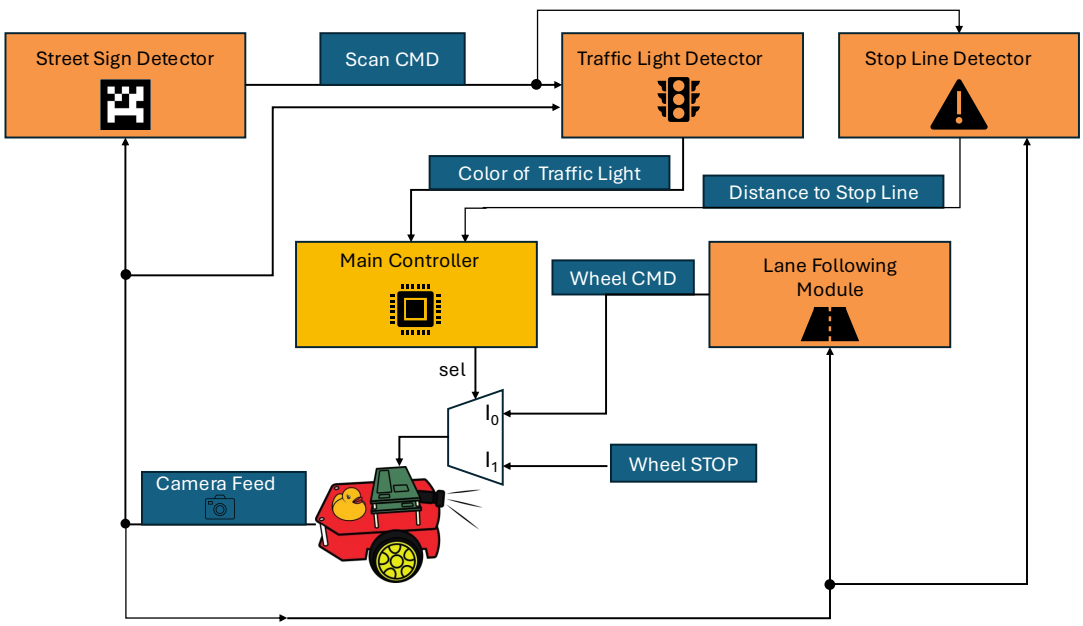

Figure 1: System Overview

The **main controller** serves as the central decision-making unit. It collects data from various detectors and modules, enabling coordinated and informed responses. Below, we describe the functionality and implementation of each module and their respective purposes.
We noticed that the Duckiebots do not have perfectly equal hardware. To counteract this effect, most of the modules have a separate configuration file that is stored as `<VEHICLE_NAME>.yaml` to allow quick and easy changing and storing different parameters for each Duckiebot.

Cédric Bugnon, Marcello Krahforst & Laurent Schroeder

1. Street Sign Detector Module

In the Duckietown Framework, each traffic sign is associated with a unique AprilTag. This is used by The *Street Sign Detector Module*, illustrated in Figure 1, which recognizes traffic light street signs by using an AprilTag Detector.

When a camera image is published, it is stored in a local variable and processed in a frequency that can be set by the user with the help of a parameter. In our tests, we used a processing frequency of 4 Hz since the Duckiebot was not able to keep up with the original frequency of 20 Hz. The detections are made with functions from the `dt_apriltags` library and whenever a tag is detected whose ID is in the correct range that indicates that it is a traffic light street sign (74 to 94), a TagInfo message containing the Tag Id is published to the ROS topic `/<VEHICLE_NAME>/traffic_signs` and a Float message carrying the distance to the AprilTag is published to `/<VEHICLE_NAME>/at_distance`.

The information provided by this module is essential for the following *Traffic Light Detector Module*, as the system only searches for traffic lights when a corresponding street sign is detected, ensuring that traffic light detection only occurs in proximity to intersections with traffic lights. This ensures both more robustness and less computational load.

2. Traffic Light Detector Module

This module uses a small computer vision pipeline. As introduced before, to optimize resource usage and computation load, the system only searches for traffic lights when in proximity to one. Upon detecting the appropriate AprilTag, the module receives a “Scan CMD” signal (as shown in Figure 1), which sets the `self.process` variable to `True` and activates the traffic light detection function.

The detection process begins by preprocessing the images: first, we undistort the images, then apply color masks creating images, filtering for the red and green traffic light colors respectively. We obtain the color ranges for the masks by first recording a ROS bag of the camera feed and using the colorspace’s program provided by the exercise 3 from this course. Ideally this is done for each Duckiebot separately, since the ranges change depending on the lighting conditions and which Duckiebot is used. These settings can be stored in a config file for each Duckiebot. Next, the `detect_traffic_light` function is called to identify the status of the traffic light. The pipeline utilizes OpenCV’s `HoughCircles` function to detect circular shapes, under the assumption that traffic lights are circular. This makes our street light detection more robust against noise from other light sources, as we only look for circular structures. To further improve robustness, the module tracks the last detected color of the traffic light. If two consecutive detections report the same color, the system considers the detection valid. Finally, the detected color is published to the `traffic_light_color` topic, to which the main controller subscribes in order to make decisions, as illustrated in Figure 1.

3. Stop Line Detector Module

The Duckiebot initially faced an issue where it would stop too early upon detecting the red light of the traffic light, rather than continuing to the designated stop line. To address this, we implemented the Stop Line Detector Module. This module is designed to accurately identify red stop lines on the road and trigger the Duckiebot to stop at the correct position.

The module operates using two primary components: image processing and decision-making. In the image processing stage, the Duckiebot's camera outputs a continuous stream of images. These images are cropped to our region of interest (in our case, we crop out 25% of the upper part) to exclude the upper portion, as the stop line is expected to appear only in the lower part of the frame. A color mask is then applied to isolate the specific shade of red corresponding to the stop line. In the initial iteration, we searched for rectangular shapes to improve robustness against noise. However this turned out to be computational expensive, which is why we switched to an average red intensity calculation. Surprisingly, this method turned out to be not only less computational expensive but also more reliable and robust.
In the decision-making stage, this average red intensity value is compared to a predefined threshold to determine whether a stop line is present. If the intensity exceeds the threshold, the system detects a stop line. We then keep track of the red line, and as soon as it is no longer visible, we command our Duckiebot to stop immediately. The stop command is published as a Boolean value via the `stop_line` publisher. When a red line is detected, the Boolean is set to `True` and published. The main controller, which will be detailed in a later section, subscribes to both the `stop_line` and `traffic_light_color` publishers. This approach ensures reliable detection and responsive stopping behaviour. However, in the end, we replaced this module with the distance to the traffic light's AprilTag, as it was more robust and less computationally expensive.

4. Lane Following Module

The *Lane Following Module* is a critical component of our system, ensuring the Duckiebot stays centered in its lane and maintains the proper position to detect traffic lights and stop lines effectively. This module utilizes the `lane_control` feature from the Duckietown framework's dt-core, with parameters tuned to optimize its performance.
While the lane detection functionality performed exceptionally well, requiring minimal adjustments to the color mask that filters out yellow and white lines, the controller initially presented challenges. The Duckiebot exhibited significant oscillations and tended to drift too far to the right of the lane centre. At times, it almost left the road entirely, losing sight of the yellow guideline.
To address this, we increased the k-factor of the integral control for both position and angular orientation, which enhanced the correction of steady-state offsets to some degree. However, the most significant improvement came from fine-tuning the d_offset parameter. This parameter specifies the desired deviation from the centre of the lane. By carefully adjusting this value, we achieved much better results, enabling the Duckiebot to follow the lane's centre more consistently and effectively.
The *Lane Following Module* sends out wheel commands to our Duckiebot (as illustrated in Figure 1), which are responsible for the main movement.

5. Main Controller Module

The *Main Controller Module* serves as the central decision-making unit of our system, with the ability to override motor commands from the *Lane Following Module*, as can be seen in Figure 1. In our initial iteration, the module integrated the outputs of both the *Stop Line Detector* and *Traffic Light Detector*, which are Boolean values. The logic was structured so that the

`stop_line` publisher outputs `True` when a red line is detected, while the `traffic_light_color` publisher outputs `False` when a red traffic light is detected. The Duckiebot is programmed to stop only when **both conditions are met**: the `stop_line` detection is `True` and the `traffic_light_color` detection is `False`. When these conditions are satisfied, the Main Controller overrides the wheel commands coming from the *Lane Following Module* and sets both wheel speeds to zero.
However, this method was not reactive enough, as the Duckiebot often stopped too late. As a result, it could no longer see the traffic light and was unable to determine when the light turned green again.
To address this issue, we switched from relying on the stop line detection to calculating the distance relative to the AprilTag. If the Duckiebot is closer than a configurable threshold, we send a stop command (if the light is red). To further enhance robustness, we used this distance measurement to determine whether the Duckiebot is directly under the light, allowing it to continue driving to avoid getting stuck. This adjustment prevents the Duckiebot from misinterpreting reflections of red from the traffic light's AprilTag as a red light.
This dual-condition approach significantly enhances robustness against uncertainties, such as light reflections. For instance, if the Duckiebot encounters a scenario where another Duckiebot's taillight appears red, it will not stop unless it is closer than a certain threshold to the traffic light's AprilTag, ensuring accurate responses only when both conditions are met.
Centralizing the logic in the Main Controller also simplifies system organization and adaptability. Each submodule, such as the *Street Sign Detector* and *Traffic Light Detector*, is responsible only for publishing its outputs, without directly interfering with other processes. This separation reduces the risk of errors and facilitates easier adjustments or improvements across individual modules.

Evaluation and Results

In this chapter, we will outline our evaluation methodology and present the corresponding results.

Systematic evaluation Method

To evaluate our system, we will test each module individually and then combine them for integrated testing.
1. Lane Following Module:
Testing the Lane Following Module is straightforward. We will assess its ability to follow straight and curved lanes of varying difficulty levels. By modifying the lane structures, we can verify whether the tuned parameters perform as expected under different conditions.
2. Traffic Light Color Detection:
In this section, we evaluate the Traffic Light Detector's ability to accurately identify traffic light colors. The test comprises three subtests designed to assess the system's robustness.
2.A) Red Light Test:
In the first subtest, the Duckiebot is positioned in front of the red line, simulating a real-world

scenario where vehicles must wait at a red light. The traffic light is initially set to red. The goal of this test is to measure how effectively and promptly the Duckiebot detects a change in the light's color and responds by moving forward when the light turns green. The experiment setup can be seen in Figure 2.

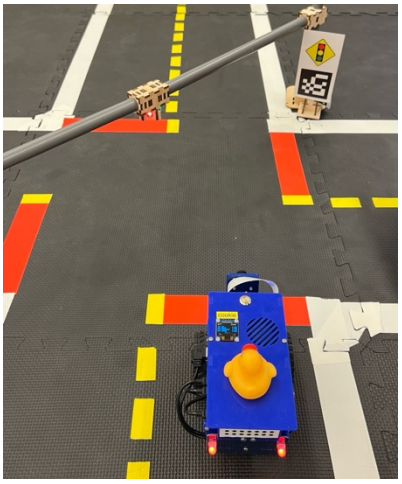

Figure 2: Red Light Test experiment setup.

2.B) Red Light Stop Test:

To further evaluate the responsiveness of our system, we tested the Duckiebot's ability to stop at a red light. In this scenario, the Duckiebot approaches a traffic light that remains red throughout the test. The system is expected to recognize the red light and stop at a predefined position. The stopping position is determined based on the measured distance to an AprilTag. If the Duckiebot detects that it is closer to the AprilTag than the set threshold, it should trigger the stop command. This ensures the Duckiebot halts at the correct position relative to the traffic light. To determine our algorithms robustness, repeated the experiment for different wheel speeds of the Duckiebot. The wheel speeds we changed by increasing the torque that each wheel is able to output.

The experiment setup can be seen in Figure 3. We have a fixed position where we let the Duckiebot drive to the traffic light.

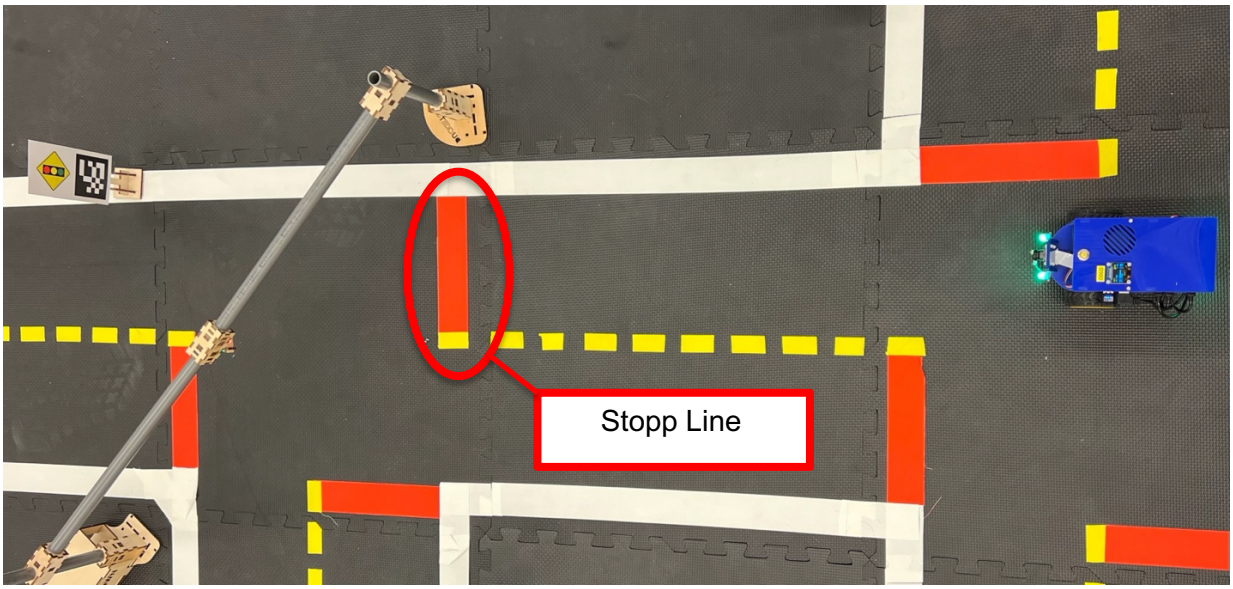

Figure 3: Red Light Stop Test experiment setup.

2.C) Green Light Test:

In this test, we aim to evaluate whether the Duckiebot can drive continuously across the intersection when the traffic light is green, without stopping midway. Additionally, we want to ensure that the Duckiebot does not mistakenly interpret the red lights from perpendicular intersections as the traffic light it should follow. In our setup, opposing traffic lights are always active. For example, when the light for our lane is green, the light for the lane to the right is red. To assess this, we set the traffic light to green and allow the Duckiebot to drive towards it. The expectation is that the Duckiebot will continue across the intersection without stopping. The experiment setup can be seen in Figure 4.

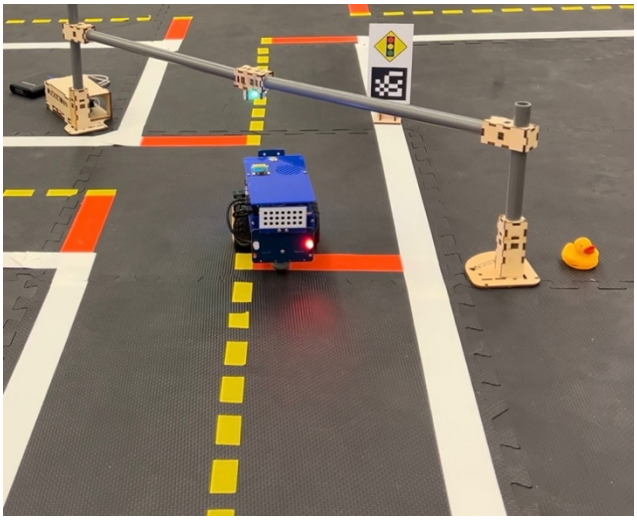

Figure 4: Green Light Test experiment setup.

3. Traffic Light Detector Distance Test:
In this experiment, we aim to stress-test our algorithm's ability to detect traffic lights at varying distances. Since the Duckiebot always approaches the traffic light from a distance and begins

searching for it well before reaching the red stop line, we now want to evaluate the maximum distance at which it can still detect traffic lights accurately.

To conduct the test, we placed the Duckiebot at different distances from the red line and measured how many times it detected a green or red light within a 10-second window. The baseline was established by placing the Duckiebot directly in front of the red line, where we counted the number of detected traffic lights. We then compared these results to those obtained from other distances. Figure 3, illustrates the experimental setup, showing the Duckiebot positioned at the maximum distance we tested.

Systematic evaluation Results

2.A) Red Light Test Results:

The results are presented in Figure 5.

- On the left, we plot the trial number against the Duckiebot's measured response times.
- On the right, we plot the distribution of response times.

The response times are centered around the mean of 1.38 seconds. In comparison, it takes a human roughly 1.25 seconds to react to a changing traffic light[1]. Our system demonstrates stability, with a standard deviation of 0.49 seconds, indicating consistent performance and no significant outliers.

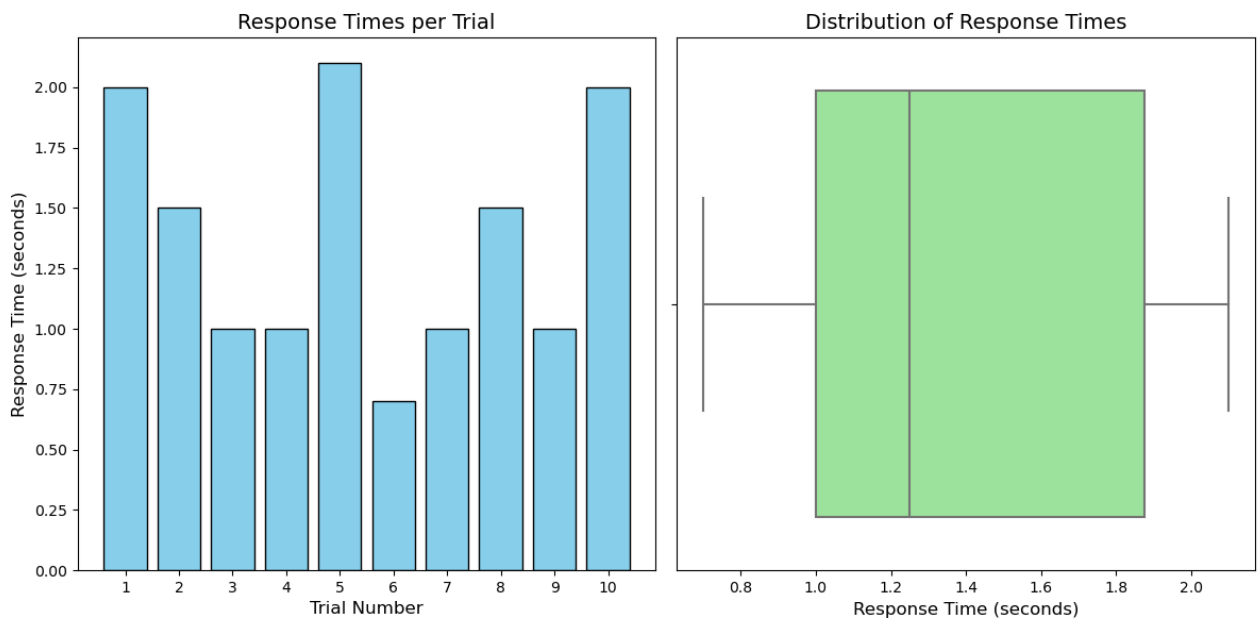

Figure 5: Response times of Red-Light Test.

2.B) Red Light Stop Test Results:

The results are presented in Figure 6.

- **Left Plot:** This graph displays the trial number on the x-axis and the Duckiebot's measured stopping distances on the y-axis. To stress-test our detection system, we

[1] Green, M. (2000). "How Long Does It Take to Stop?" Methodological Analysis of Driver Perception-Brake Times. *Transportation Human Factors*, *2*(3), 195–216. https://doi.org/10.1207/STHF0203_1

conducted the experiment at five different velocities. The ideal stopping position is represented by a dotted line. A negative stopping distance indicates that the Duckiebot overshot the stop line, while a positive value suggests the Duckiebot stopped too early. Trials where the Duckiebot failed to stop at the stop line are marked with a colored cross corresponding to the specific velocity.

- **Right Plot:** The distribution of the stopping distances across all trials is shown in this plot. This helps visualize how the Duckiebot's stopping behavior varies across the different velocities.

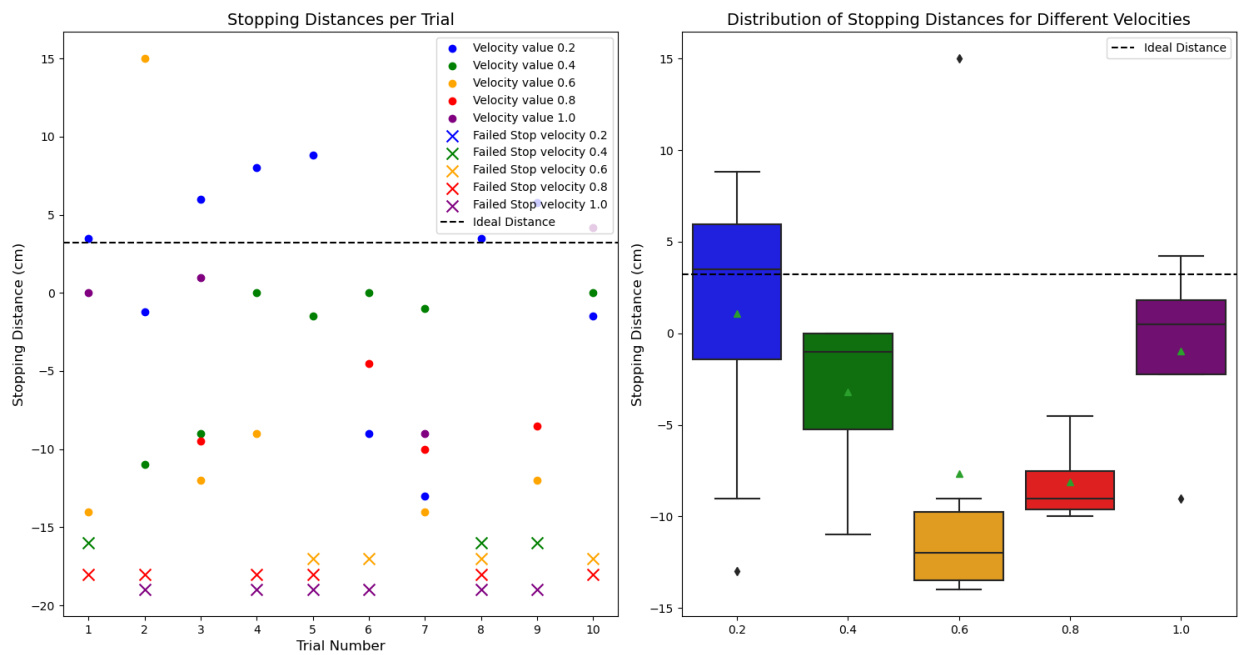

Figure 6: Distances to stop line for the Red Light Stop Test.

We observe that at a velocity value of **0.2**, the stopping distance is centered around the ideal stopping point. This outcome is expected and desired, as the Duckiebot has more time to react to the traffic light at lower speeds. The mean stopping distance is **1.09 cm**, which is close to the ideal stop distance. This distance can be fine-tuned by adjusting the position of the AprilTag, as it depends on the relative distance between the Duckiebot and the AprilTag. As the speed increases, we notice that the average stopping distance from the ideal line also increases. This is intuitive because the Duckiebot has less time to react to the traffic light at higher speeds. Additionally, the Duckiebot requires more time to come to a complete stop, so it travels farther before halting.

Interestingly, the distribution of stopping distances, shown in the right plot, narrows as the speed increases. This might seem counterintuitive but can be explained by the fact that more trials had to be excluded at higher speeds, as the Duckiebot failed to stop entirely. Consequently, the distribution reflects only the trials where the Duckiebot successfully stopped, which aligns with the increasing number of failed trials visible in the left plot.

Overall, we can conclude that the Duckiebot demonstrates a degree of robustness to changes in speed, also performing well at a velocity value of **0.4**. However, beyond this point, the likelihood of the Duckiebot failing to stop increases significantly, as the algorithm requires more time to detect the red light.

2.C) Green Light Test Results:
In all ten test cases, we observed that the Duckiebot successfully approached the green light and crossed the intersection without stopping or encountering any issues.

3. Traffic Light Detector Distance Test Results:
The test results are shown in Figure 7, where we plot the number of detected green and red lights against the distance at which the Duckiebot was placed from the stop line.

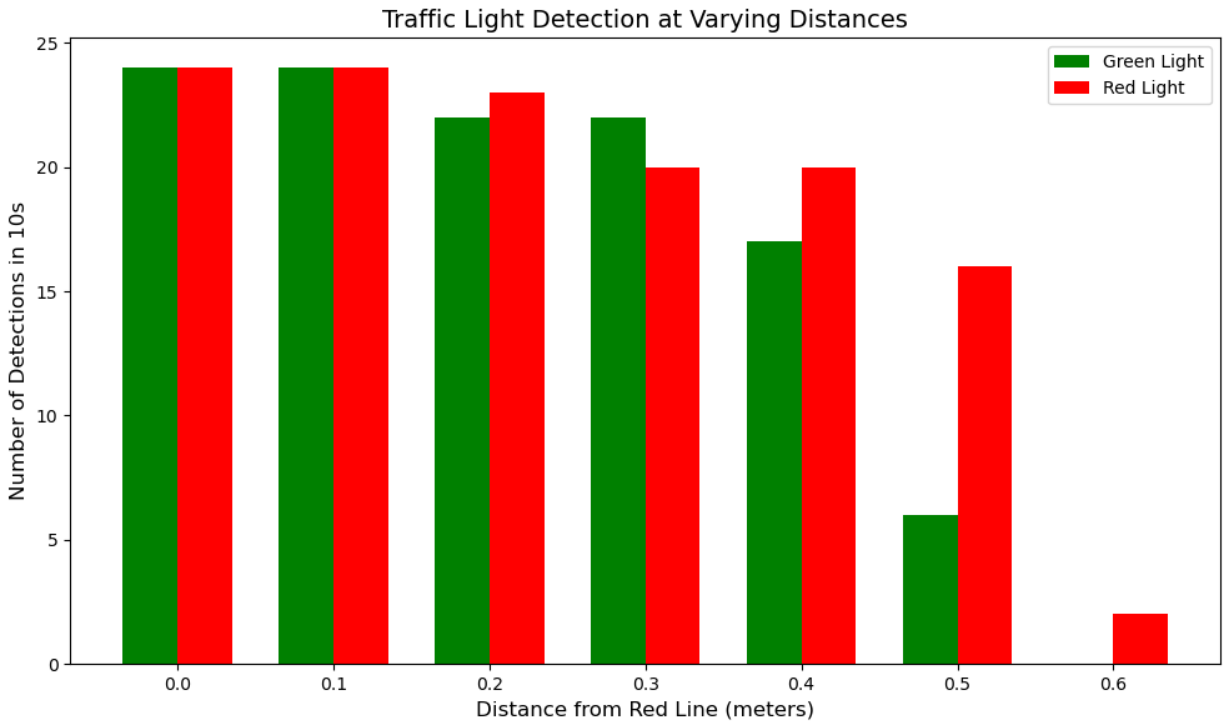

Figure 7: Traffic Light Detector Distance Test Results.

As expected, we observe that as the Duckiebot's distance from the stop line increases, the number of detected red and green lights decreases. A notable finding is that the red-light detection remains accurate over a longer distance compared to the green-light detection. This could be due to the superior color filter used in our red-light detection system.

Next Steps

In this chapter, we will discuss how this project could be extended and what we would change if we were to start the project again.

1) Future Work

There are several areas where this project could be extended. To begin, one could implement functionality to determine the direction the Duckiebot should take when crossing an intersection

with a traffic light. This could be achieved with minimal effort, as we already know how to override commands from the lane following and detect when the Duckiebot is crossing a traffic light. Furthermore, this decision-making process could be enhanced using a shortest-path planning algorithm, allowing the Duckiebot to navigate to a desired destination optimally. However, this feature would require substantial effort, as it necessitates global knowledge of the map and awareness of the decisions available at each point on the map.
Another potential feature is the ability to handle scenarios where duckies cross the street despite their signal being red. The Duckiebot could respond appropriately by stopping, even if its traffic light is green. This feature could be implemented relatively quickly, requiring only a filter to detect duckies and an extension to the controller to monitor and react to such events.
During the presentations from the other teams, one team mentioned that they used GPU acceleration to process the images with a neural network. Offloading the image manipulations done by the modules to a GPU might provide a boost in performance and reduction in CPU load that would be beneficial for running the street sign detection with the lane following module from dt-core in the background. Thus, it would be worthwhile to investigate that optimization.

2) Considerations

If we were to restart the project from scratch, we would implement the lane-following functionality ourselves. During the project, we realized that the *Lane Following Module* from dt-core consumed significant CPU resources, preventing other modules from running at higher frequencies, which are crucial for a smoother and more robust solution. Although we attempted to eliminate some unnecessary computations in the existing *Lane Following Module*, we believe it would have been simpler to develop our own implementation. Overall, we should have thought more about the computational limitations of the Duckiebots and made our code more efficient from the ground up as well as noticing sooner that the computational load of lane following might cause problems.
The project was overall well organized, and the workflow was mostly well structured. We managed to maintain the git repo in a good state and were successful in modularizing the work in a way that all members of the team could work on the project simultaneously.

Retrospective

In this chapter, we will give a reflection on our project outcome compared to the planned project.

Project Scope

The project scope was limited to the minimum required outcome specified in our project outline. This was mainly caused by the roadblocks we encountered at the end of the project regarding the CPU load when running all the modules together with the lane following module from dt-core. Nonetheless, the minimum requirements, which stated that we would have a working traffic light detection system that stops and starts the lane following, were fulfilled. We also briefly tested the system with more than one traffic light in Duckietown and came to the conclusion that it would work since the system rejects traffic lights that are further away.

However the success is limited by the badly working lane following and crossing of road crossings.

Approach

Our approach changed many times due to the challenges we faced along the way. The initial plan was to implement a traffic light detection system that would only work based on the traffic signs and the traffic lights which ironically is fairly similar to the final outcome. However, after some testing we noticed that the Duckiebots were not halting in a good position at the intersection and were rather stopping whenever they see a red light. This is why we implemented the stop line detector, first utilizing the openCV approxPolyDP function in order to detect the red line at the intersection. However, when finally testing the entire system with the lane following from dt-core, we noticed much slower reaction times and delays compared to when running each element separately. We narrowed the problem down to the CPU not processing everything fast enough. The first suspect was the *Lane Following Module* from dt-core since it starts ten nodes and many visualizations. The suspicion was correct since that module alone used about 70% of the CPU load. We tried slowing all modules down and chose 4 Hz since it seemed to be fast enough for the system to run in real-time. Besides that, we scaled the images down to a lower resolution to decrease the computation load. Nonetheless, this did not reduce the CPU load to a point which allowed the system to run in real-time. We further simplified the stop line detector to only calculate the average pixel intensity at the bottom of the image with a color range isolation filter applied. This was also not efficient enough thus, in a final attempt, we decided to omit the stop line detector entirely and only rely on the distance to the AprilTag to stop at the correct position. This makes the system faster on the one hand, but on the other hand, it is less robust since it strongly depends on the position of the street sign. Nonetheless, it showed promising results and worked pretty accurately with the dt-core lane following. In parallel to this, we also tried using a lane-following module provided by our mentor to decrease the CPU load. This lane following module was much simpler than the dt-core lane following and only relied on the color intensities on the left and right side of the road and controls the steering based on the position of the intensities in the image. This drastically reduced the CPU load, thus making stopping in the correct position in front of the traffic light more reliable. However, it also decreased the performance of the lane following. In the end, we let the user choose which lane following to use by choosing the appropriate branch (main for the dt-core lane-following and enjie_lanefollowing for our lane-following implementation strongly inspired by our TA Enjie).

Results

The results of our traffic light detection pipeline work as expected. However, the lane following, which was expected to work out of the box and was not part of the project, caused a lot of problems and did not work to a degree that could be considered satisfactory in the end. Nonetheless, the project can be considered successful since the basic requirements of our initial commitment were fulfilled.

Considerations

The project was fun, and the exercises were helpful in preparing us for our project.

Github Repository

Link
To clone: git clone https://github.com/ETHZ-DT-Class/DT-Project-LightGuard.git